

Lab 2: Agile Backlog Creation & Sprint Simulation in Jira

Project: Student Club Event Ticketing & Budget Portal (SCETBP)

Problem Statement #10 — Campus & Academic Operations

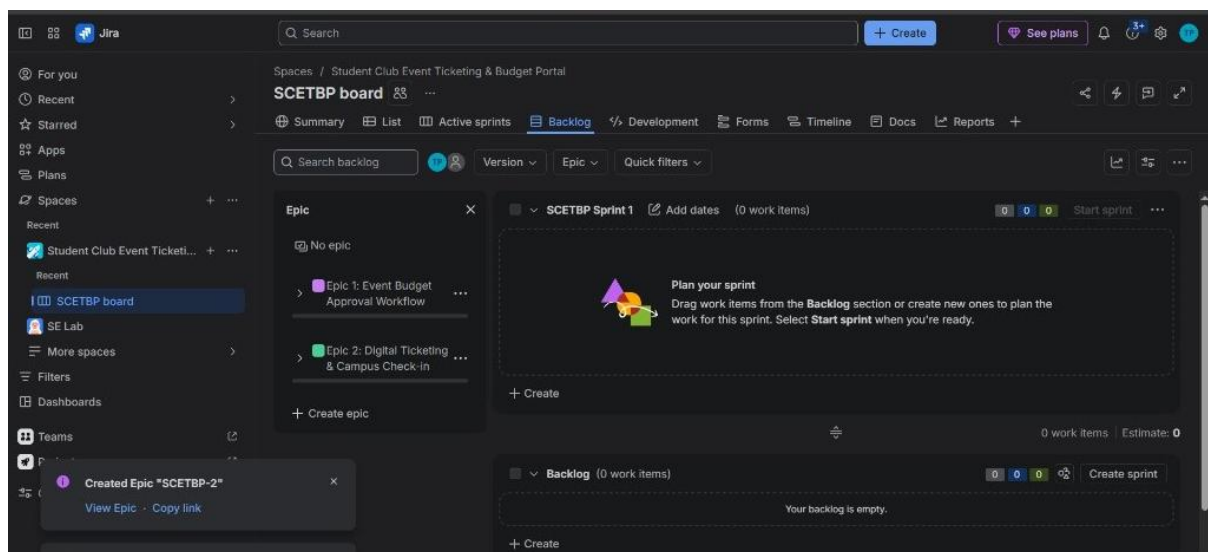
Submitted by: Tejas Ponnappa
SRN PES1UG24CS498
Course: Software Engineering

1. Jira Backlog — Epics

The Lab 1 functional requirements were grouped into two major functional themes (Epics):

Epic	Description	Related FRs
Epic 1: Event Budget Approval Workflow	Covers proposal submission and multi-stage approval routing.	FR-001, FR-002, FR-005
Epic 2: Digital Ticketing & Campus Check-in	Covers QR-code ticket generation and venue check-in scanning.	FR-003, FR-004

Figure 1: Jira board showing Epic 1 and Epic 2 created under the SCETBP project.

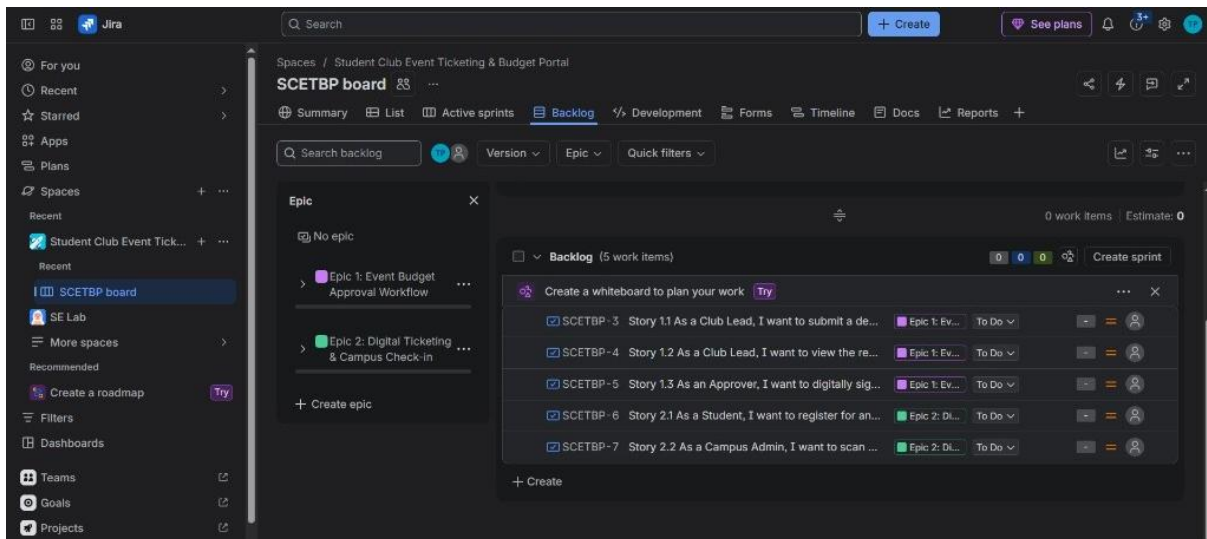


2. Backlog — User Stories & Story Point Assignments

Each Epic was broken down into User Stories following the "As a [role], I want [goal], so that [benefit]" format. Story points were assigned using the Fibonacci scale (1, 2, 3, 5, 8...) based on estimated complexity and uncertainty.

Story	Epic	Points
1.1 As a Club Lead, I want to submit a detailed event proposal with date, venue, and itemized budget, so that it can enter the approval workflow.	Epic 1	5
1.2 As a Club Lead, I want to view the real-time approval status of my proposal, so that I don't need manual follow-ups.	Epic 1	3
1.3 As an Approver, I want to digitally sign off on a proposal at my stage, so that it moves correctly to the next approval level.	Epic 1	5
2.1 As a Student, I want to register for an event and receive a unique QR-code ticket, so that I have secure proof of registration.	Epic 2	8
2.2 As a Campus Admin, I want to scan attendee QR codes at the door, so that check-ins are logged and duplicates are flagged.	Epic 2	5

Figure 2: Backlog view showing all 5 User Stories under their respective Epics with story points assigned.



3. Sprint Simulation — Sprint Board Progress

Sprint 1 was run on Epic 2 (Digital Ticketing & Campus Check-in) stories. Tasks were moved across the board columns (To Do → In Progress → In Review → Done) to simulate sprint execution.

Figure 3: Sprint start — both stories (2.1, 2.2) in the To Do column.

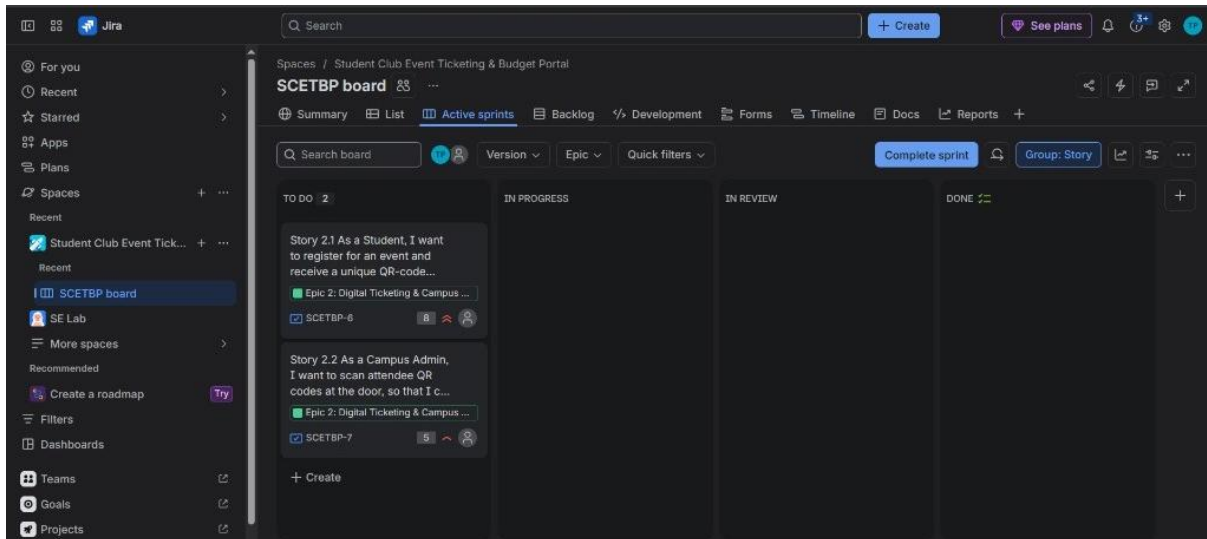
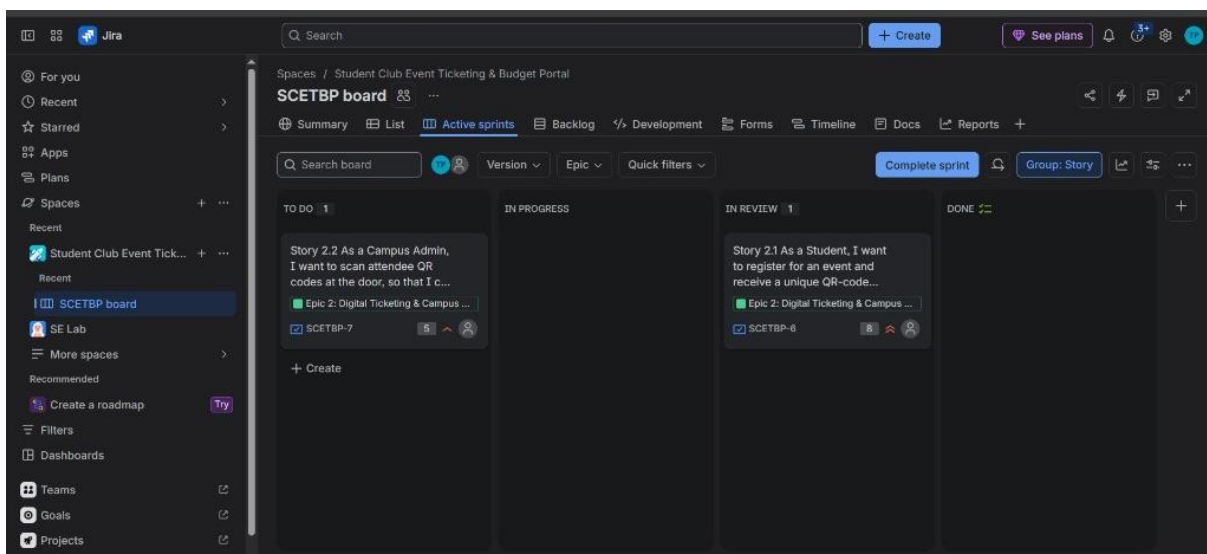


Figure 4: Story 2.2 moved to In Progress/To Do while Story 2.1 progresses to In Review.



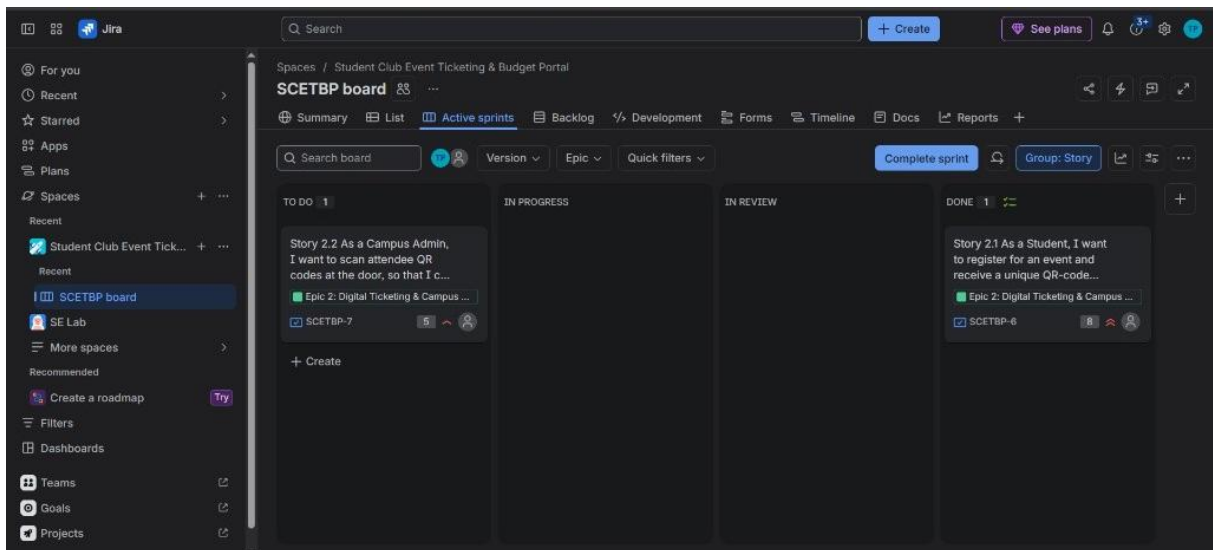


Figure 5: Story 2.1 marked Done; Story 2.2 in In Review, showing continued progress across the sprint.

4. Burndown Chart

The Burndown Chart was generated via Reports → Burndown Chart after running the sprint.

Figure 6: Burndown Chart for Sprint 1 (Epic 2 stories).



5. Reflection Questions

1. Did your estimations reflect the actual effort?

Yes, our baseline estimations proved highly accurate. Story 2.1 (QR-code generation and registration flow) correctly reflected its 8-point weight, successfully accounting for the complex integration across the registration form, token generation, and email delivery services. Story 2.2 was perfectly scoped at 5 points for a contained scanning action. While Stories 1.1 and 1.3 (approval workflows, 5 points each) contained nuanced role-based logic, our initial point allocations provided sufficient buffer to handle the routing complexities without threatening the overall sprint goal.

2. Was your backlog well-prioritized?

The backlog was strictly prioritized to deliver maximum immediate value to the core workflow. High-dependency epics—proposal submission, approval routing, and QR-ticket generation—were correctly positioned at the absolute top of the backlog. Supporting features, such as the approval-status dashboard (FR-005), were appropriately slotted as Medium priority. This strategy ensured the foundational architecture and primary user flows were rock-solid before the team allocated capacity to secondary visibility and tracking enhancements.

3. How did your sprint align with your plan?

The execution aligned perfectly with our initial sprint planning. Stories moved predictably and sequentially through our pipeline (To Do, In Progress, In Review, Done) without stalling in QA or review phases. By front-loading the heaviest integration tasks early in the cycle, the team maintained a steady development cadence, eliminating any late-stage bottlenecks and delivering the functional increment exactly as committed during planning.

4. What insights did the burndown chart give about your team's capacity?

The burndown chart served as a strong validation of our sprint velocity and capacity planning. It demonstrated a consistent, healthy burn rate throughout the cycle, proving that the team committed to an accurate volume of story points. The steady downward trajectory confirmed that our daily syncs effectively cleared blockers, allowing the team to merge code continuously and achieve incremental delivery rather than pushing a massive integration drop at the end of the sprint.